

Sérialisation avancée

Depuis le début de la session nous envoyons et recevons des données au format JSON. Cependant, si vous souhaitez envoyer des images ou des fichiers, le format JSON n'est pas adapté. Nous allons voir comment envoyer des données binaires (images, fichiers) via une requête HTTP.

Il existe plusieurs façons d'envoyer des données binaires, mais la plus courante est d'utiliser l'objet `FormData` en JavaScript. Cet objet permet de construire un ensemble de paires clé/valeur représentant les champs d'un formulaire, y compris les fichiers. À noter que `FormData` utilise le format `multipart/form-data` pour l'envoi des données et que vous ne pouvez pas utiliser les requêtes `PUT` ou `DELETE` avec `FormData`, seulement `POST` ou `GET`.

On récupère les données du formulaire en JavaScript avec l'objet `FormData`, puis on les envoie au serveur via une requête `fetch`. Du côté serveur, on peut récupérer les données du formulaire en PHP via les superglobales `$_POST` et `$_FILES`.

1. Récupérer les données du formulaire en JavaScript et les envoyer au serveur

```
const form = document.querySelector("form");
form.addEventListener("submit", async (event) => {
    event.preventDefault(); // Empêcher le rechargement de la page
    const formData = new FormData(form); // Créer un objet FormData à partir du
    formulaire

    try {
        const reponse = await fetch("URL vers le serveur", {
            method: "POST",
            body: formData, // Envoyer l'objet FormData dans le corps de la
            requête
        });

        const donnees = await reponse.json();
        //Si la réponse n'est pas ok, on lance une erreur
        if (!reponse.ok) {
            throw new Error(donnees.message || "Erreur lors de l'envoi du
            formulaire");
        }

        // Logique pour notifier l'utilisateur du succès
        // Ex: new Toast("Message de succès", "success");
    } catch (error) {
        console.error(error);
        // Logique pour notifier l'utilisateur de l'erreur
        // Ex: new Toast(error.message, "error");
    }
});
```

2. Récupérer les données du formulaire en PHP

```

<?php
// Vérifier si le formulaire a été soumis

$nom = filter_input(INPUT_POST, 'nom', FILTER_SANITIZE_SPECIAL_CHARS);
$email = filter_input(INPUT_POST, 'email', FILTER_VALIDATE_EMAIL);
$fichier = $_FILES['fichier'] ?? null;

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // Traiter les données du formulaire
    if ($nom && $email && $fichier && $fichier['error'] === UPLOAD_ERR_OK) {
        // Déplacer le fichier téléchargé vers un répertoire de destination
        $destination = 'uploads/' . basename($fichier['name']);

        if (move_uploaded_file($fichier['tmp_name'], $destination)) {
            echo json_encode(['message' => 'Fichier téléchargé avec succès', 'nom'
=> $nom, 'email' => $email]);
            exit;
        } else {
            echo json_encode(['message' => 'Erreur lors du déplacement du
fichier']);
            exit;
        }
    } else {
        echo json_encode(['message' => 'Données invalides ou fichier manquant']);
        exit;
    }
} else {
    echo json_encode(['message' => 'Méthode HTTP non autorisée']);
    exit;
}

```

Stratégie pour la gestion des images et fichiers

Pour être efficace dans la gestion des images et fichiers, une bonne stratégie est de d'enregistrer les fichiers dans un répertoire spécifique sur le serveur et de stocker uniquement le chemin d'accès ou l'URL du fichier dans la base de données. Cela permet de réduire la taille des données stockées dans la base de données et facilite la gestion des fichiers.

Assurez-vous que la base de données contient une colonne pour stocker le chemin ou l'URL du fichier.

```

$fichier = $_FILES['image'] ?? null;
$id = $_POST['id'] ?? null;

if (!$fichier || !isset($id)) {
    throw new Exception("ID manquant ou données invalides");
}

$id = (int) $id;

```

```

$dossierImages = __DIR__ . '/../../assets/img/';
$destination = $dossierImages . basename($fichier['name']);

// Déplacer le fichier téléchargé vers le dossier des images
if (!move_uploaded_file($fichier['tmp_name'], $destination)) {
    throw new Exception("Erreur lors de l'upload de l'image");
}

$scheminRelatif = 'assets/img/' . basename($fichier['name']);

// Préparer la requête UPDATE
$requete = $connexion->prepare("
    UPDATE piscine
    SET image = :image
    WHERE id = :id
");

// Lier les paramètres
$requete->bindParam(':id', $id);
$requete->bindParam(':image', $scheminRelatif);

// Exécuter la requête
$requete->execute();

if ($requete->rowCount() > 0) {
    $reponse = ["success" => true, "message" => "Image ajoutée avec succès"];
    header("Content-Type: application/json");
    http_response_code(200); // OK
    echo json_encode($reponse);
    exit;
} else {
    header("Content-Type: application/json");
    http_response_code(404); // Not Found
    $reponse = ["success" => false, "message" => "Aucune modification effectuée
(ID peut-être inexistant)"];
    echo json_encode($reponse);
    exit;
}

```

Du côté client, vous pouvez ensuite utiliser l'URL ou le chemin d'accès stocké dans la base de données pour afficher l'image ou le fichier.

```

```

Exemple complet

```

<?php
//Afficher les erreurs-enlever en production
ini_set('display_errors', 1);

```

```

ini_set('display_startup_errors', 1);
error_reporting(E_ALL);

require_once __DIR__ . '/../config.php';

try {
    // Connexion à la base de données
    $connexion = new PDO("mysql:host=" . DB_HOST . ";dbname=" . DB_NAME . ";port="
        . DB_PORT, DB_USER, DB_PASSWORD);

    // Vérifier que la requête est en POST
    if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
        throw new Exception("Méthode non autorisée, utilisez POST");
    }

    $fichier = $_FILES['image'] ?? null;
    $id = $_POST['id'] ?? null;

    // Vérifier que le fichier et l'ID sont présents
    if (!$fichier || !isset($id)) {
        throw new Exception("ID manquant ou données invalides");
    }

    // Vérification du type de fichier pour accepter seulement les images
    $finfo = finfo_open(FILEINFO_MIME_TYPE);
    $mime = finfo_file($finfo, $fichier['tmp_name']);
    finfo_close($finfo);
    if (!in_array($mime, ['image/jpeg', 'image/png', 'image/gif', 'image/webp']))
    {
        throw new Exception("Type de fichier non autorisé. Seules les images JPEG,
        PNG, GIF et WEBP sont acceptées.");
    }

    ///////////////
    // Préparer le chemin de destination
    $id = (int) $id;
    $dossierImages = __DIR__ . '/../assets/img/';
    $destination = $dossierImages . basename($fichier['name']);

    // Déplacer le fichier téléchargé vers le dossier des images
    if (!move_uploaded_file($fichier['tmp_name'], $destination)) {
        throw new Exception("Erreur lors de l'upload de l'image");
    }

    $cheminRelatif = 'assets/img/' . basename($fichier['name']);

    // Préparer la requête UPDATE
    $requete = $connexion->prepare("
        UPDATE piscine
        SET image = :image
        WHERE id = :id
    ");

    // Lier les paramètres

```

```

$requete->bindParam(':id', $id);
$requete->bindParam(':image', $cheminRelatif);

// Exécuter la requête
$requete->execute();

// Vérifier si une ligne a été modifiée
if ($requete->rowCount() > 0) {
    $reponse = ["success" => true, "message" => "Image ajoutée avec succès"];
    header("Content-Type: application/json");
    http_response_code(200); // OK
    echo json_encode($reponse);
    exit;
} else {
    header("Content-Type: application/json");
    http_response_code(404); // Not Found
    $reponse = ["success" => false, "message" => "Aucune modification
effectuée (ID peut-être inexistant)"];
    echo json_encode($reponse);
    exit;
}
} catch (Exception $e) {
    $reponse = ["success" => false, "message" => $e->getMessage()];
    header("Content-Type: application/json");
    http_response_code(500);
    echo json_encode($reponse);
}

```